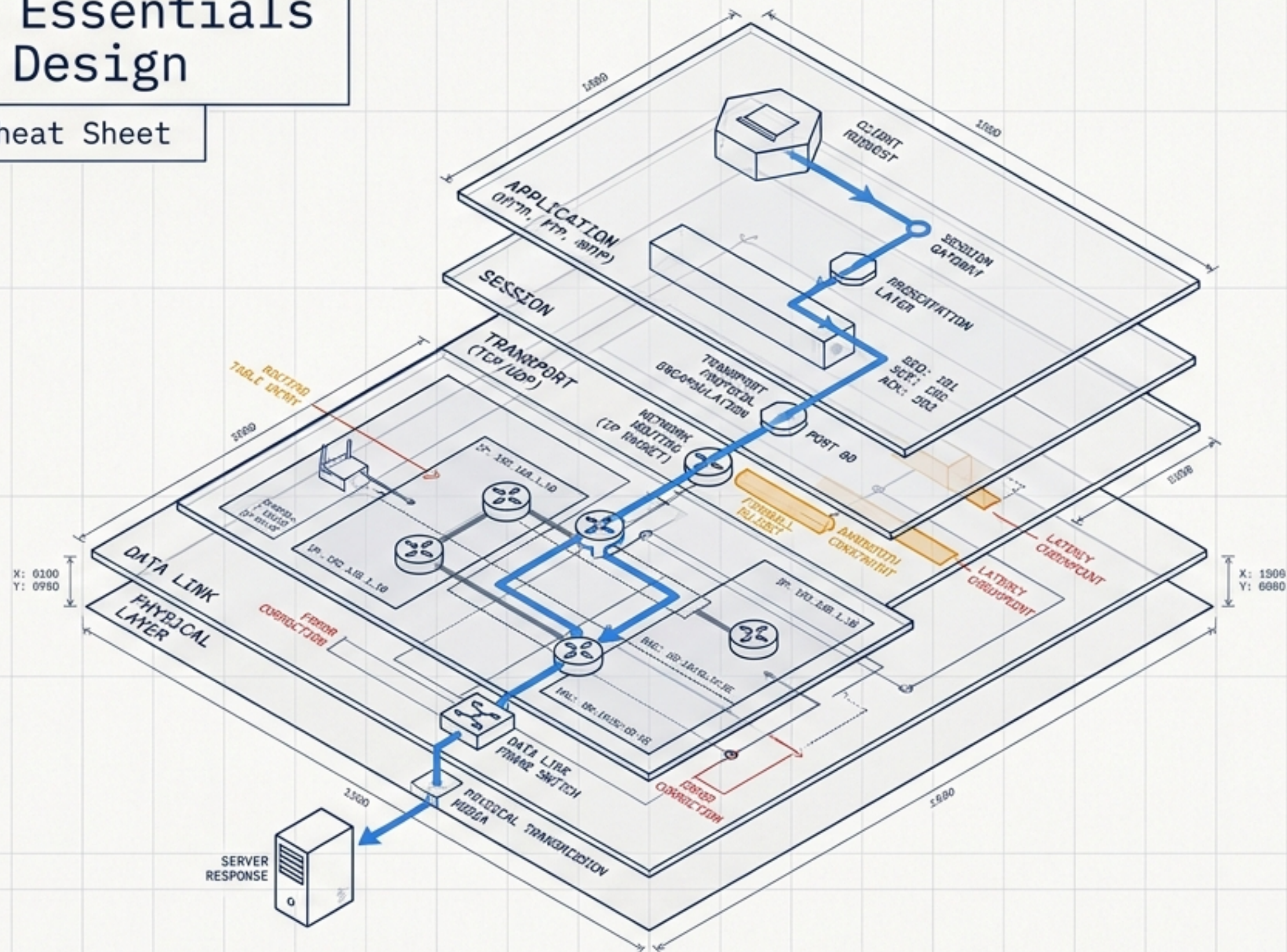


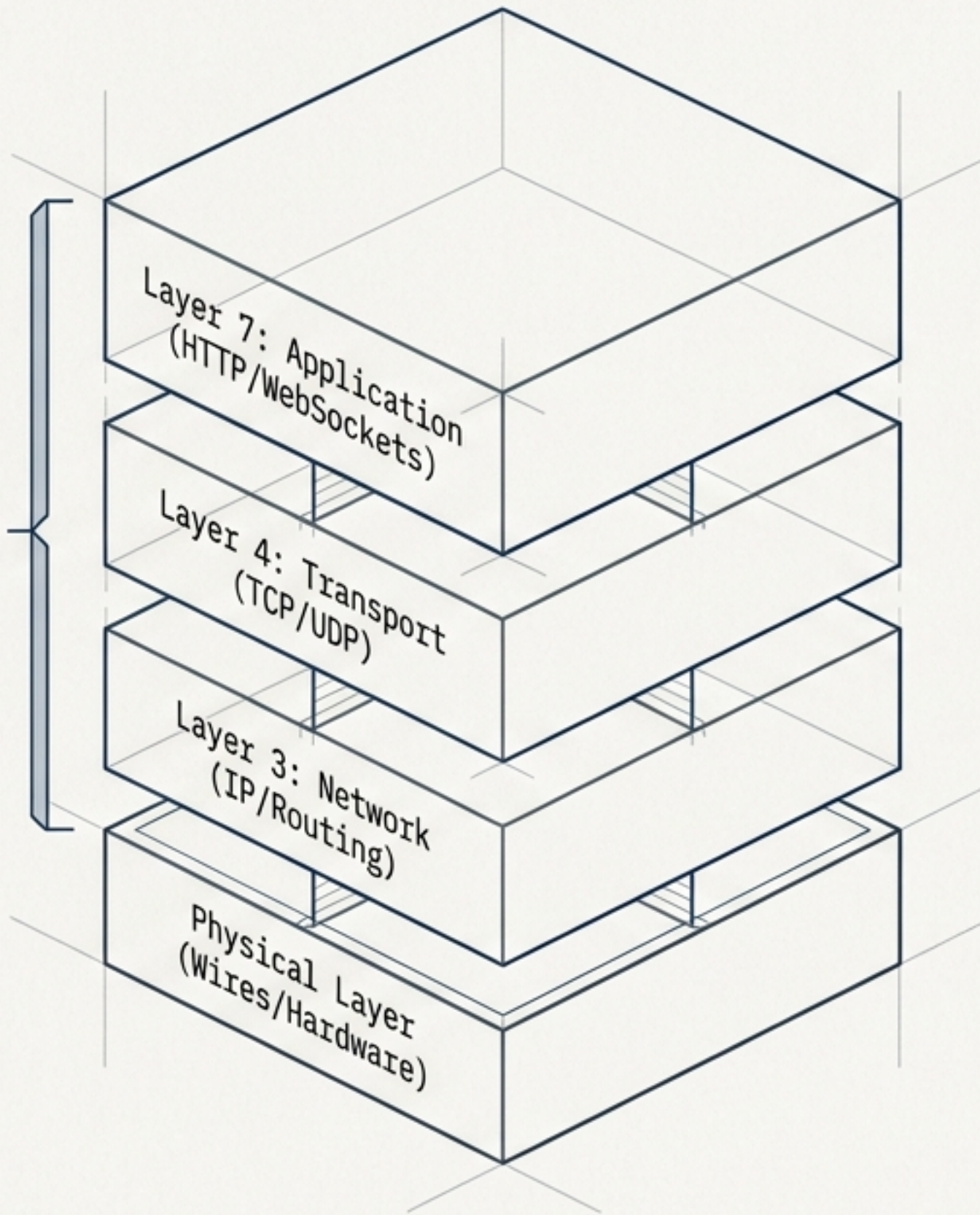
Networking Essentials for System Design

The Architect's Cheat Sheet



— DATA FLOW PATH ○ NETWORK NODE ■ SECURITY CHECKPOINT → TECHNICAL ANNOTATION

THE SYSTEM DESIGN ZONE



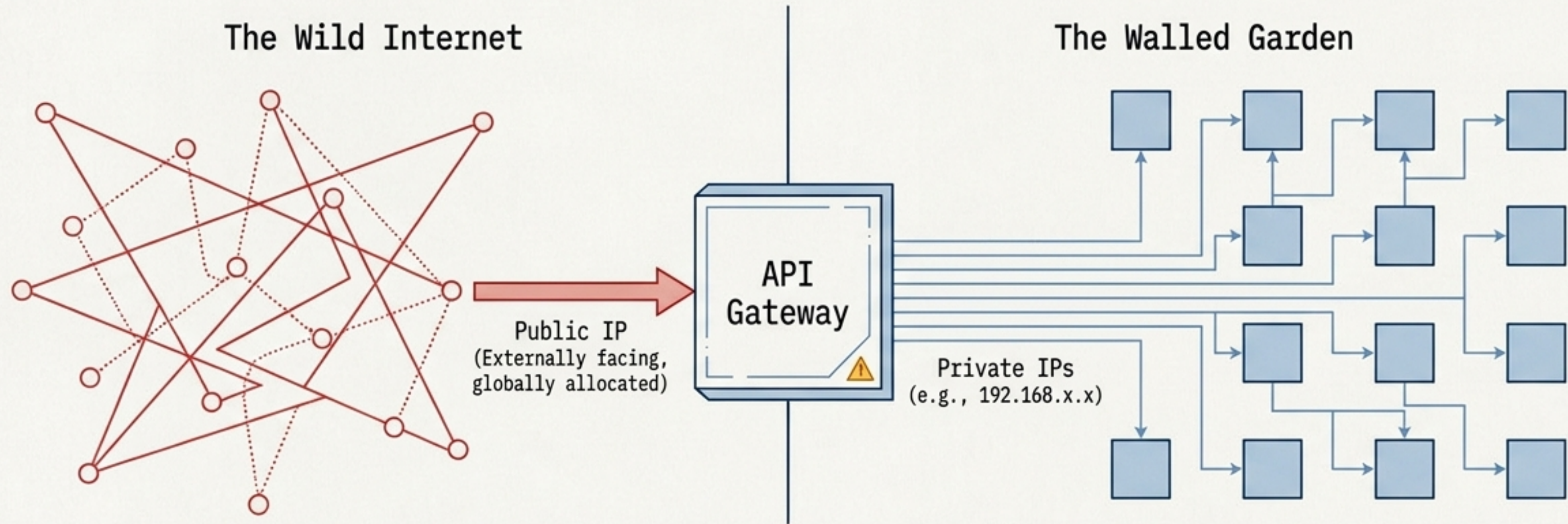
The Abstraction Principle

Higher layers assume the competence of lower layers. HTTP assumes TCP handles the handshake; TCP assumes IP routes the packets correctly.

The Inherent Trade-off

Every layer traversed upward adds developer functionality (routing → reliability → application logic) but introduces inherent physics costs: network latency and connection statefulness.

Layer 3 (Network) – Identity & Routing



IPv4 vs. IPv6

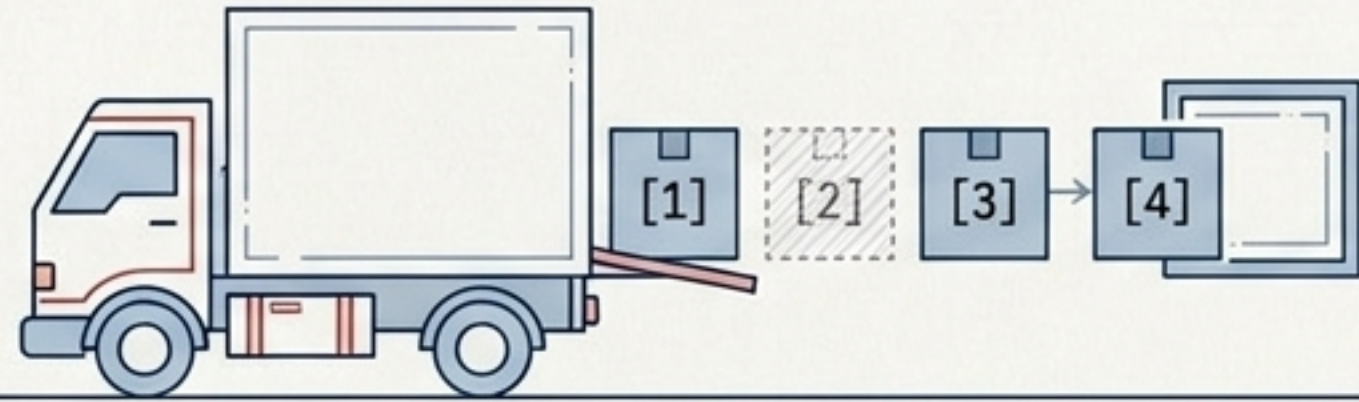
IPv4: 4 bytes, depleted, external default for maximum compatibility.
IPv6: 16 bytes, modern, preferred for internal routing.

Interview Heuristic

Never expose internal microservices to the public internet. Use private IPs internally to avoid IP overlap and secure the perimeter.

Layer 4 (Transport) – The Reliability Trade-off

TCP (The Default)

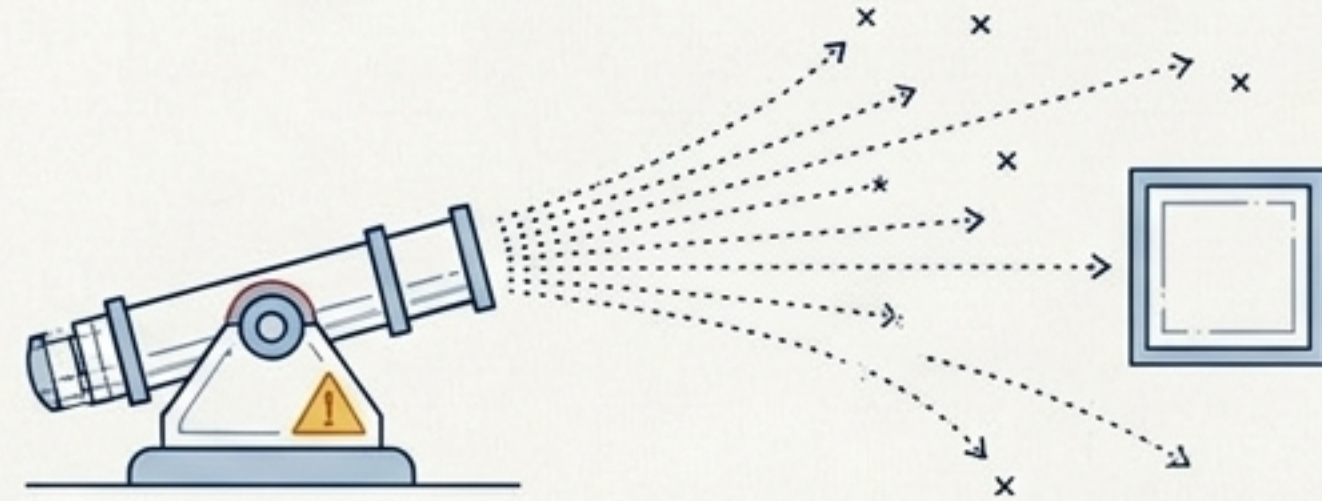


Guarantees: Reliable delivery, exact ordering.

Overhead: High (3-way handshake required).

Use Case: Web, Databases, standard API calls.

UDP (The Edge Case)



Guarantees: None (lossy, spray-and-pray).

Overhead: Minimal (stateless, extremely fast).

Use Case: Live video conferencing, multiplayer gaming.

The Interview Default: Assume TCP for 99% of system design. Only pivot to UDP when latency is paramount and dropping frames or packets is acceptable.

Layer 7 (Application) – The Restful Default

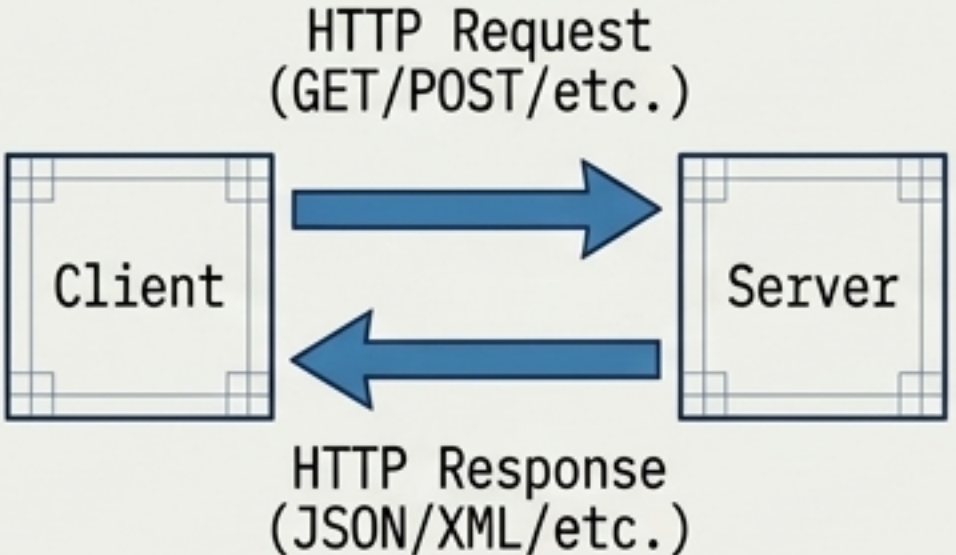
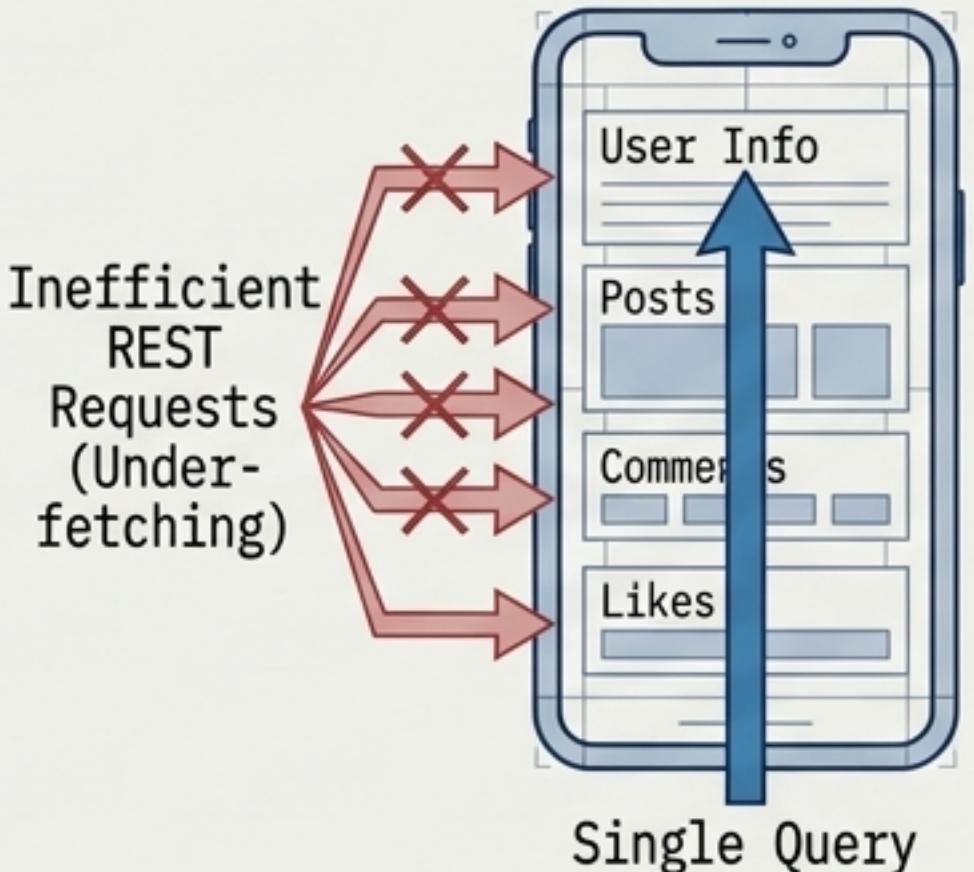
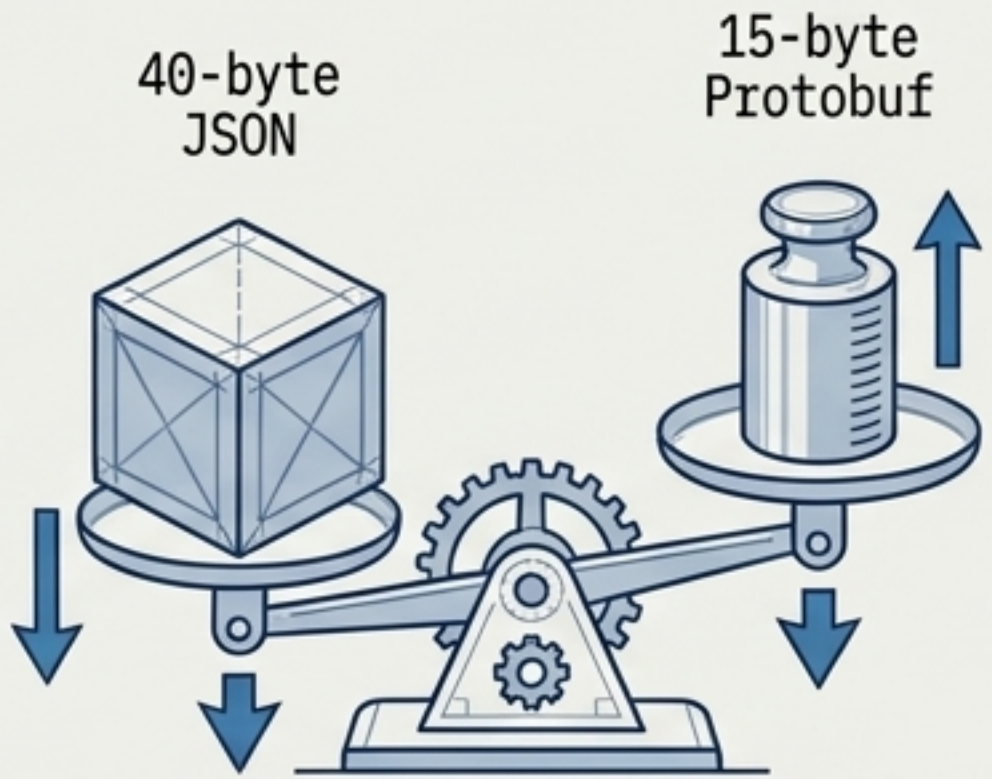
HTTP Verb (Action) + Resource URL (Noun) = Outcome

1	GET	/users/1	→ Retrieve User 1 (JSON Payload)
2	POST	/users	→ Create a new User
3	PUT	/users/1	→ Overwrite User 1 entirely
4	PATCH	/games/42	→ Update game status from stopped to running

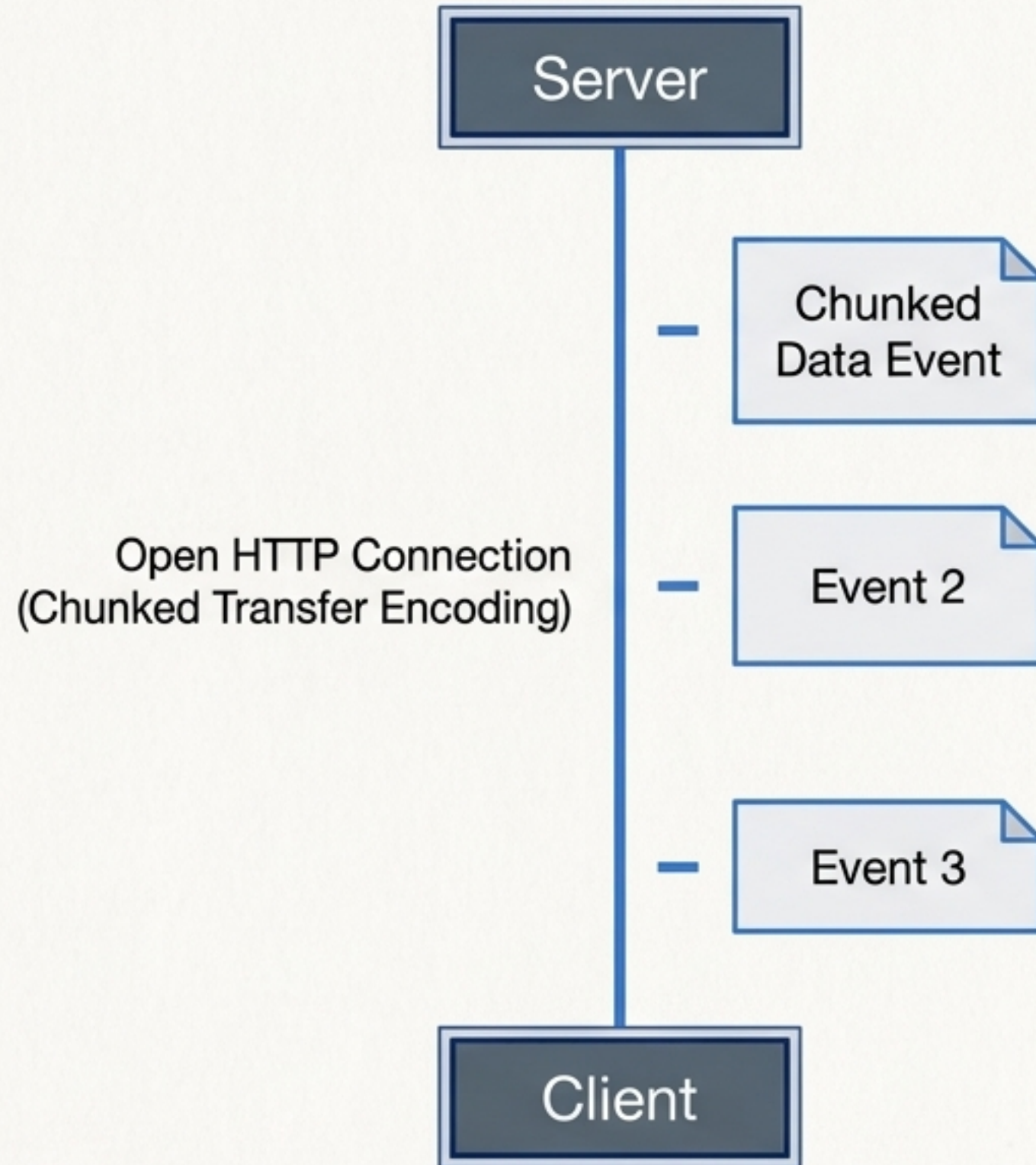


Architectural Rule: REST models Resources, not Functions. An endpoint like /update_user is an anti-pattern. You must strictly map HTTP actions to resource mutations.

Layer 7 Alternatives – Breaking the REST Default

REST (The Jack of All Trades)	GraphQL (The Flexible Frontend)	gRPC (The Microservice Muscle)
		
Verdict: Wide generality, highly compatible, reasonable performance. The safe default.	Verdict: Use when frontend requirements change rapidly or clients need disparate data structures without altering backend APIs.	Verdict: Up to 10x throughput. Use exclusively for internal server-to-server communication where extreme performance is required.

Layer 7 Real-Time – Pushing State (SSE)



The Mechanism

Server-Sent Events (SSE) leverages standard HTTP machinery using "**chunked**" encoding headers. The server pushes asynchronous updates over a single, open connection without the client needing to repeatedly poll.

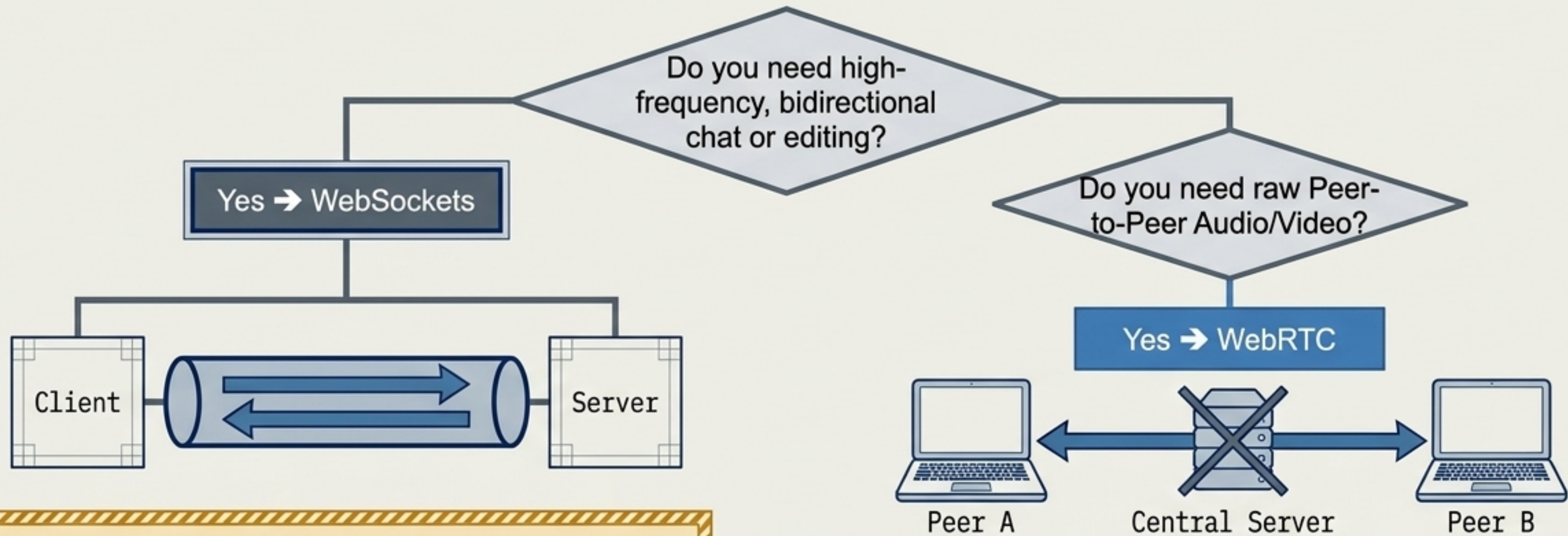
Use Cases: Stock tickers, UI progress bars, GenAI token streaming.

! The Architectural Catch

Intermediary proxies inherently sever long-running HTTP connections (typically after 30-60 seconds).

Robust SSE implementations require auto-retry logic passing the last-received Event ID to avoid dropping data during inevitable reconnections.

Layer 7 Real-Time – Bi-directional Communication

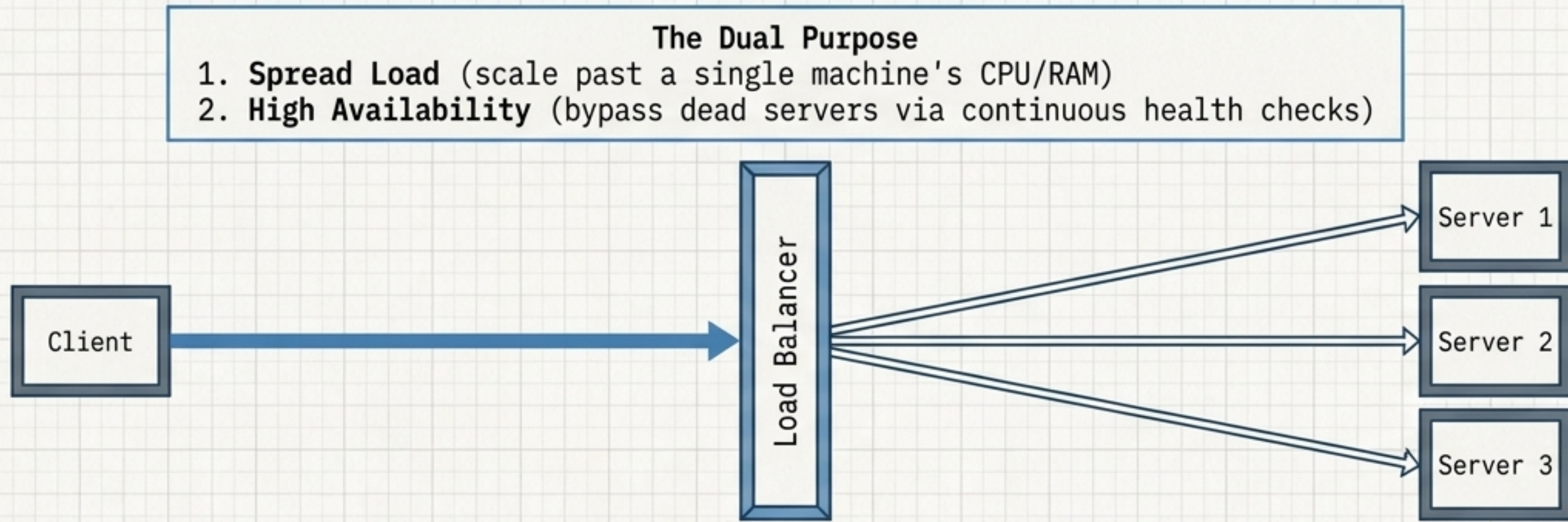


Caution: WebSockets require heavy server state. An hour-long connection locks server resources for an hour. Requires dedicated edge message brokers to scale.

Mechanism: Runs over UDP. Requires central **Signaling/STUN** servers purely to facilitate NAT hole-punching to establish the P2P link.

Architectural Rule: Do not default to WebSockets for simple, low-frequency updates. Statefulness destroys horizontal scaling simplicity.

Scalability – The Load Balancing Mandate



Client-Side Load Balancing

- Client queries a registry and routes itself.
- **Pros:** 0 extra network hops (lowest latency).
- **Cons:** Cannot update instantly (DNS caching takes minutes to hours).
- **Best for:** Internal microservices (like gRPC).

Dedicated LB Appliance

- A hardware or software middleman intercepts traffic.
- **Pros:** Real-time dynamic routing, highly configurable.
- **Cons:** Adds 1 extra network hop.
- **Best for:** The standard for all external internet traffic.

Load Balancing Deep Dive – Layers & Algorithms

Layer 4 LB (Transport)



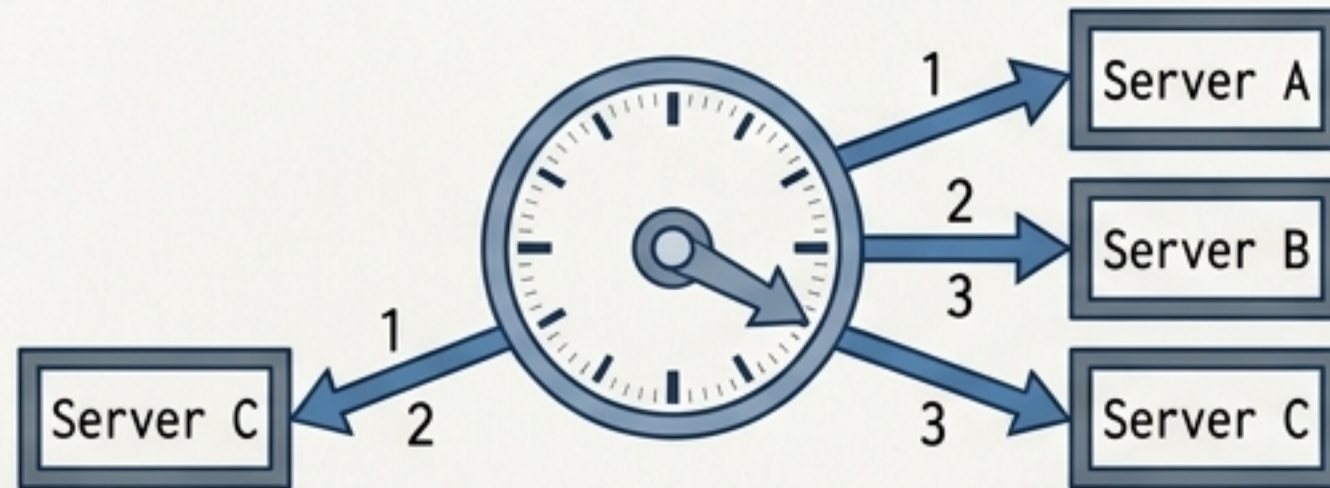
Blind TCP passthrough. Extremely fast. Essential for stateful, long-lived connections like WebSockets.

Layer 7 LB (Application)



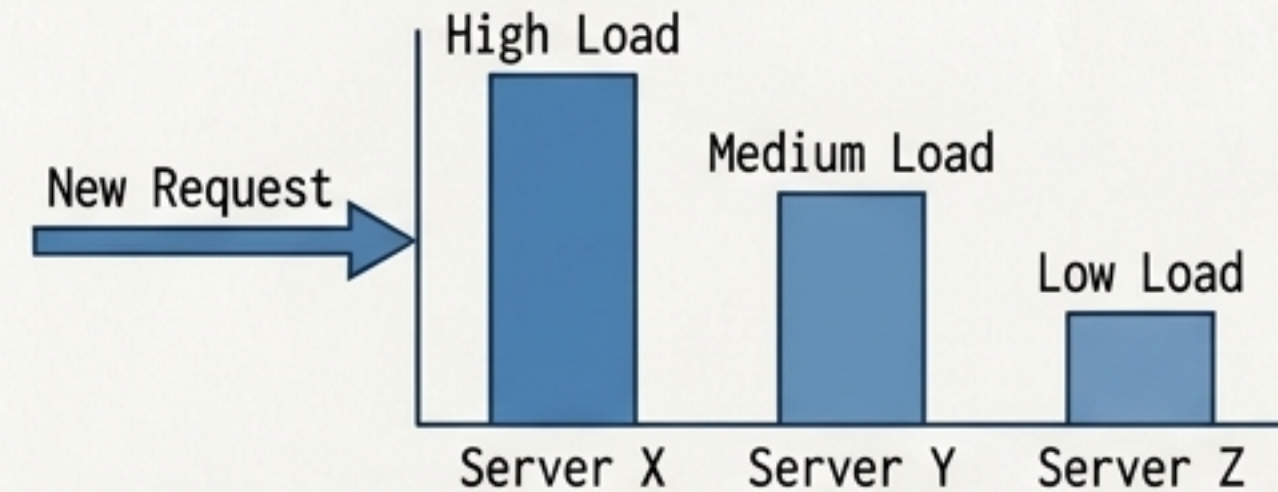
Terminates connection, inspects HTTP headers/URLs, makes smart routing choices. The default for web APIs.

Algorithm: Round Robin



Sequential dealing. Best for stateless, standard HTTP web traffic where request durations are uniform and predictable.

Algorithm: Least Connections



Routes traffic to the server with the lowest active load. Critical for long-running connections to prevent heavily skewed server burnout.

Global Scale - Beating the Speed of Light



Colocation Strategy

Data and processing must be regionally partitioned and colocated.

You cannot place the Web Server in London and the Database in NY; the multi network hops will compound latency and destroy performance.

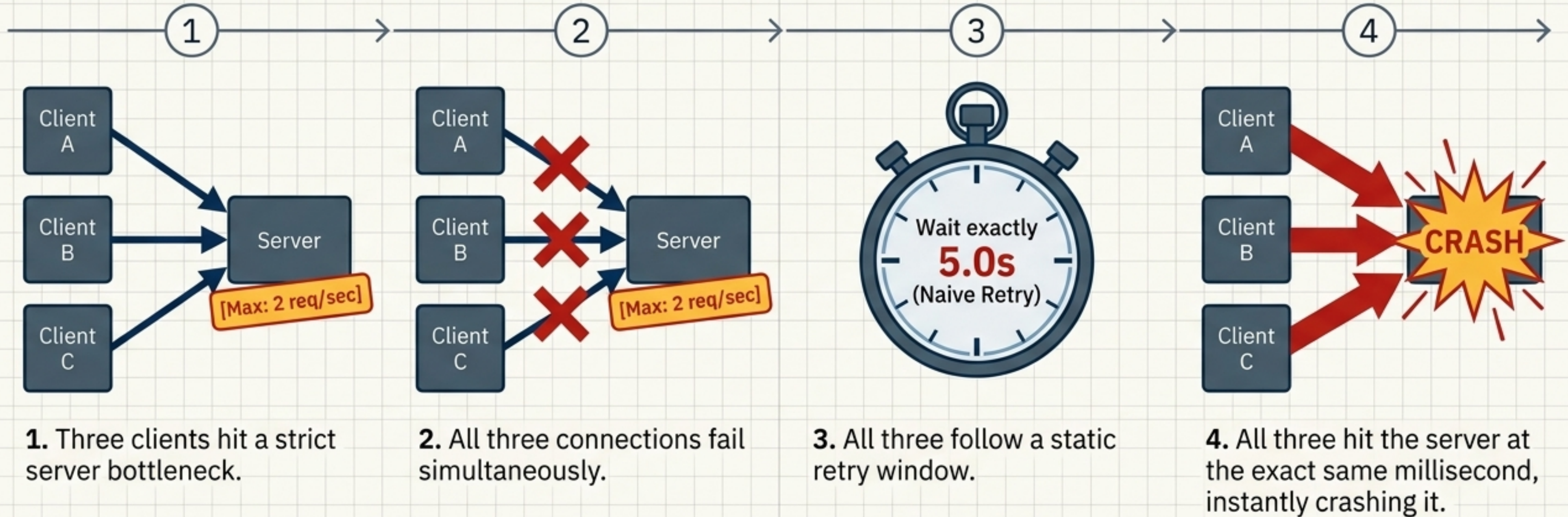
CDNs (Edge Caching)

Content Delivery Networks dot the globe with proxy servers.

Serve static content directly from the user's local city node, totally bypassing the origin server to achieve zero-hop delivery.

Surviving Failure – Timeouts & The Thundering Herd

The Synchronization Problem



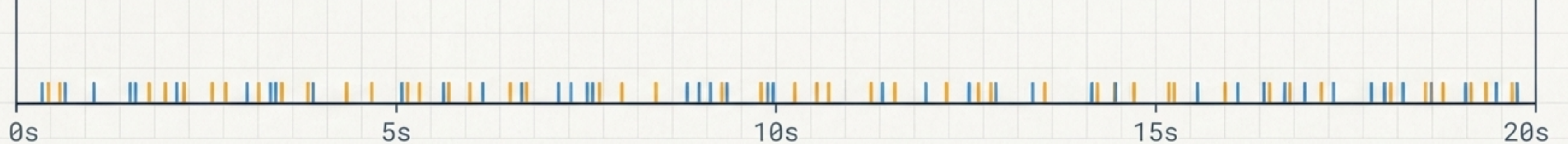
Key Insight: While **Timeouts** are mandatory to prevent hanging threads, **static retry windows synchronize** individual client failures into **massive, localized DDoS spikes**.

The Gold Standard – Exponential Backoff + Jitter

The Problem: Clustered Retries



The Solution: Distributed Load



Formula: Exponential Backoff + Jitter

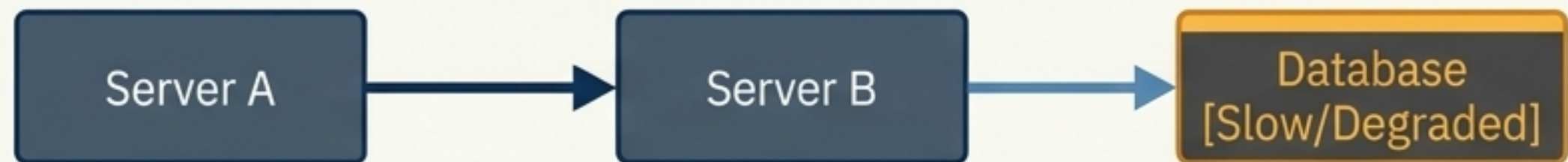
Exponential Backoff:

Wait progressively longer between retries (e.g., 2s, 4s, 8s). This prevents spamming and gives a struggling server vital breathing room to heal.

Jitter (Randomness):

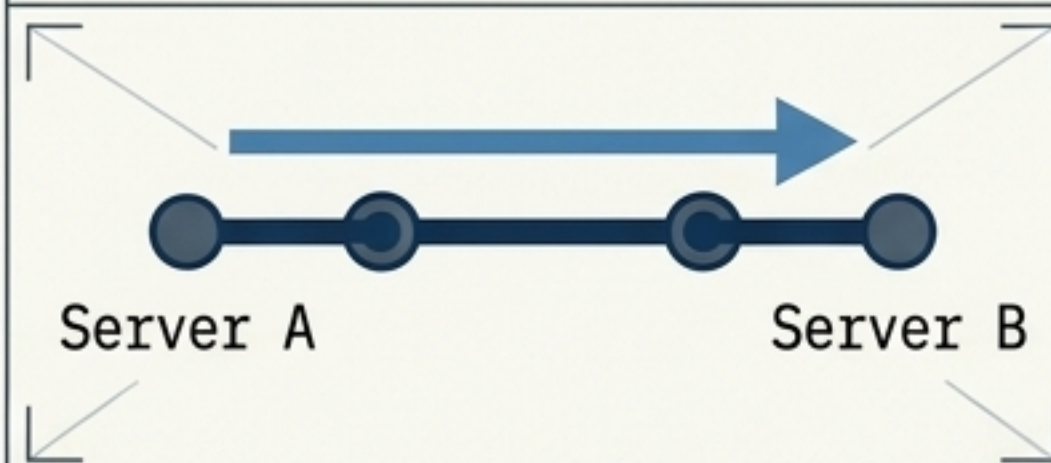
Add a randomized time offset to each client (e.g., Client A waits 2.5s, Client B waits 1.9s). This breaks mathematical synchronization and smooths overall system load.

Systemic Protection – The Circuit Breaker



The Danger: Server A repeatedly calls Server B, but B is hanging waiting for a slow DB. Server A exhausts all its own threads and crashes. A cascading failure.

Closed State (Normal Flow)



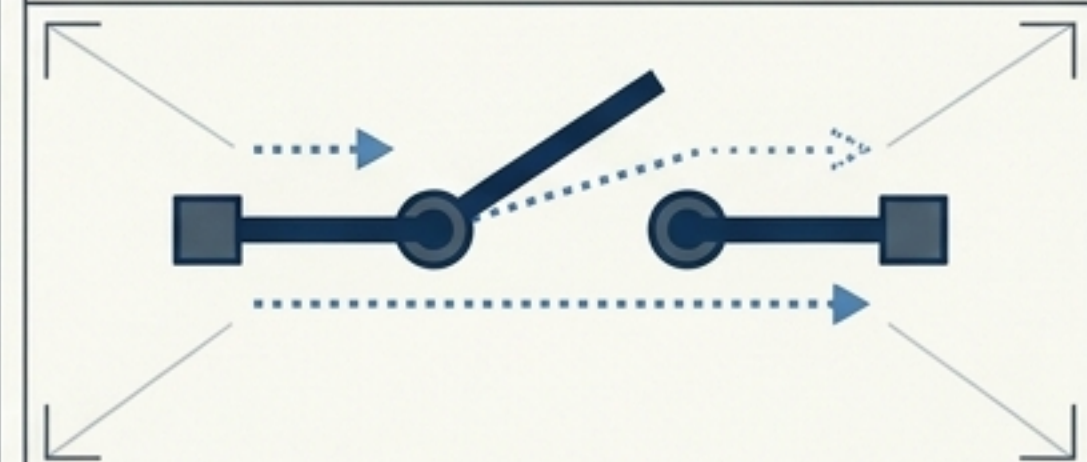
Error rates are low. Traffic flows normally.

Open State (Tripped / Failing Fast)



Error threshold exceeded. Breaker trips. Server A immediately fails requests locally without sending traffic, protecting its own threads and giving Server B time to recover.

Half-Open State (Testing Recovery)



Periodically sends a single test ping to check if Server B has recovered. If successful, it fully closes; if it fails, it trips back open.

Synthesis – The Architect's Toolkit

The Baseline (Start Here)

Safe default answers for interviews.

Transport	TCP (Reliable, ordered delivery)
API Layer	HTTP / REST (Resource-based, versatile)
Load Balancing	Dedicated L7 Appliance + Round Robin
Resilience	Strict Timeouts + Exp. Backoff with Jitter

The Pivots (Edge Cases)

When physics or state demand it.

Transport	UDP (Raw latency, A-V streaming)
API Layer	gRPC (Internal throughput), GraphQL (Fluid frontends)
Load Balancing	L4 Appliance + Least Connections (For stateful websockets)
Real-Time	SSE (Push), WebSockets (Chat), WebRTC (P2P Video)

The Golden Rule: The best architects don't just memorize the tools; they know exactly when system trade-offs demand a departure from the default.